

MediaForge

Download. Convert. Share as a link.

A full-stack media platform delivered through a website and a Telegram bot, built on one shared Python core. This document walks through what it does, how it is designed and the engineering decisions behind it.

Abhay Sharma
Full-stack web and bot platform
Python, Flask, PostgreSQL, Docker

CONTENTS

What is inside

Executive summary	3
The problem and the goals	4
The website	5
Downloader	6
Converter	7
File to link	8
Accounts and sign-in	9
Mobile layout	10
The Telegram bot	11
Tiers, limits and fairness	12
Architecture	14
Request lifecycle	16
Queue and priority	17
Protecting the host	18
Extraction and sizing	19
Source reliability	20
Data model	22
Storage and delivery	23
Security and privacy	24
Admin and operations	25
Deployment	26
Testing and verification	27
Problems found and how they were solved	28
Design decisions	29
One download, end to end	30
Technology stack	31
Glossary	32
Limitations, stated plainly	33
Skills this project demonstrates	34
Thank you for reading	35

OVERVIEW

Executive summary

MediaForge turns a pasted link or an uploaded file into a finished result, and keeps working fairly when many people use it at once.

MediaForge is a web application and a Telegram bot that share a single Python codebase. A person can paste a public media link and get the file in the quality they choose, convert their own video, audio or image into another format or size, or turn a file into a temporary shareable link. The same account, limits and history work on both the website and Telegram.

The project is small enough to understand in an afternoon and deliberately built like a service that has real users: accounts and email verification, membership tiers, daily quotas, bans, an admin console, background job queue, disk protection, structured logging, automated backups and a deployment pipeline.

> **How to read this document.** Sections 1 to 3 explain the product. Sections 4 to 12 explain the engineering. The final sections cover testing, honest limitations and the skills this project demonstrates. Screens come from the real templates and style sheets; figures shown in them are sample data.

At a glance

Area	What it delivers
Interfaces	Responsive website (dark and light theme) and a button-driven Telegram bot, both calling one service layer.
Sources	Eleven platform families: YouTube, Instagram, Facebook, Threads, Dailymotion, TikTok, X, Vimeo, Reddit, SoundCloud and Terabox.
Conversion	Video, audio and image conversion with format, resolution, aspect ratio, frame rate, bitrate and target size options.
Fairness	Three tiers, pooled daily limits, a priority queue with visible positions, and per-feature file size caps.
Safety	SSRF-aware URL validation, CSRF protection, rate limits, hashed passwords, ban lists and a strict content security policy.
Operations	Docker image, managed Postgres, health endpoints, scheduled maintenance, backups, IST-aware admin tools.

Scale: about 10,500 lines of application Python, 16 database tables, about 60 routes and about 135 automated test functions.

CONTEXT

The problem and the goals

Most downloader sites are a single text box wrapped in ads. The interesting engineering is everything around that text box.

Online media tools tend to fail in the same ways: they break when a source changes, they let one heavy user slow everybody down, they silently run out of disk, and they say "error" without saying why. MediaForge started as an attempt to build the opposite: a tool that explains itself, degrades gracefully and treats operations as part of the product.

Goals

- **One core, two front doors.** Anything the website can do, the bot can do, using the same rules and the same account.
- **Predictable under load.** Work waits in an ordered line with a visible position instead of failing when the server is busy.
- **Honest errors.** Every failure maps to a short, plain-language message and a decision about whether it counts against a daily limit.
- **Protect the host.** Disk, memory and concurrency are bounded by design, since the production container is small.
- **Operable by one person.** Admin tools, backups, logs and health checks live inside the product.

Non-goals

- Bypassing DRM, logins or paywalls. Only public media is supported, and the terms say so.
- Being a general file host. Shared links expire and files are removed on a schedule.
- Horizontal scale. The design targets a single replica and says so openly (see Limitations).

Constraints that shaped the design

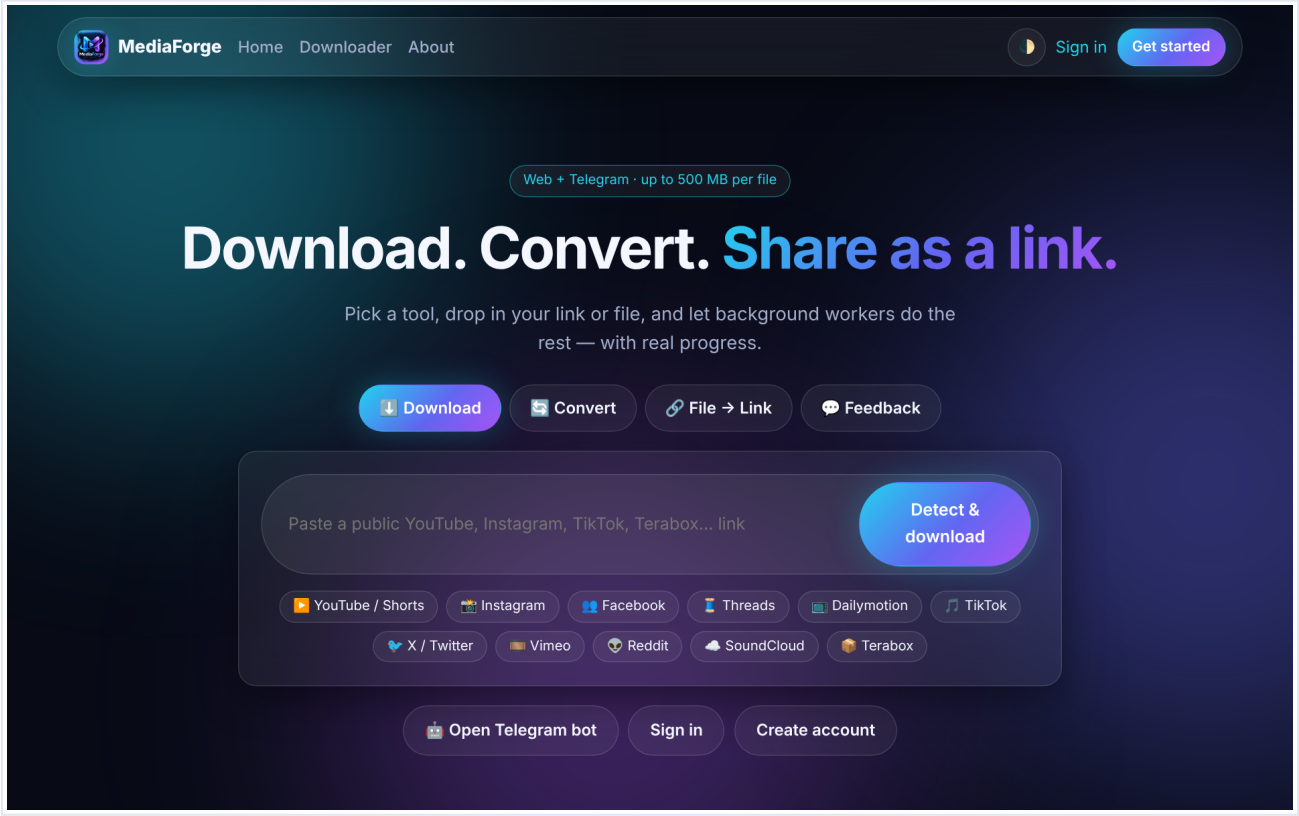
Constraint	Consequence
Small ephemeral disk on the host	A disk guard with reservations, and finished files moved to Telegram storage.
Datacenter IP address	Some sources challenge such addresses, so extraction has retries, cooldowns and optional operator levers.
Free-tier friendly hosting	One process runs web server, bot and workers; a keep-alive monitor must stay cheap.
Users in India	Daily limits reset at midnight IST and every timestamp is displayed in IST, while the database stays in UTC.

PRODUCT TOUR

The website

A single-page-feel interface with a glass-style dark theme, a light theme and a layout that works on phones.

Every page below is rendered from the application's own templates and style sheets. Quota counters and community statistics are placeholders because the pages were captured without a database.



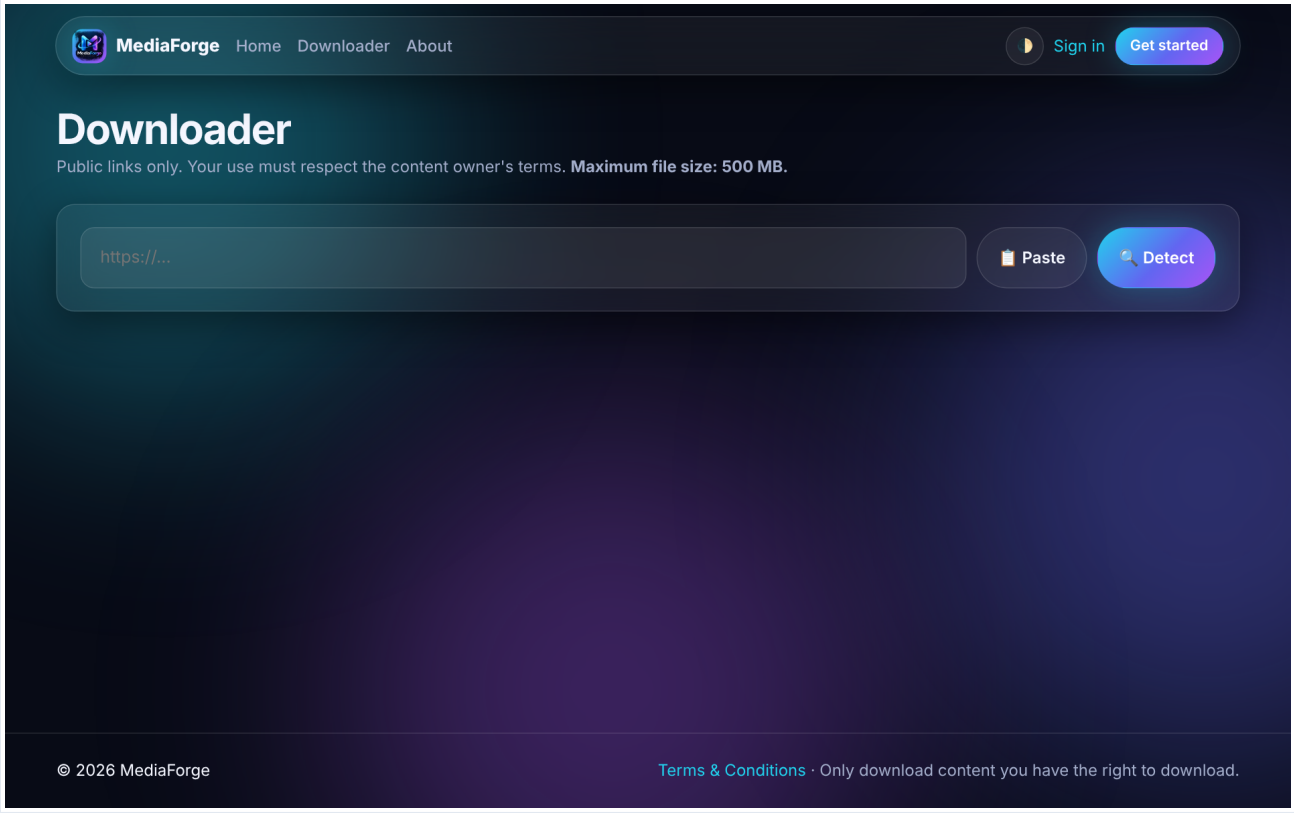
Home: pick a tool, paste a link, and see the supported platforms. The badge states the signed-in user's file-size limit.

The home page doubles as the downloader entry point. Pasting a link and pressing the main button opens the downloader with the link pre-filled and the lookup started automatically, so there is no second paste.

PRODUCT TOUR

Downloader

Detect the platform, show real choices with their sizes, then run the job with live progress.



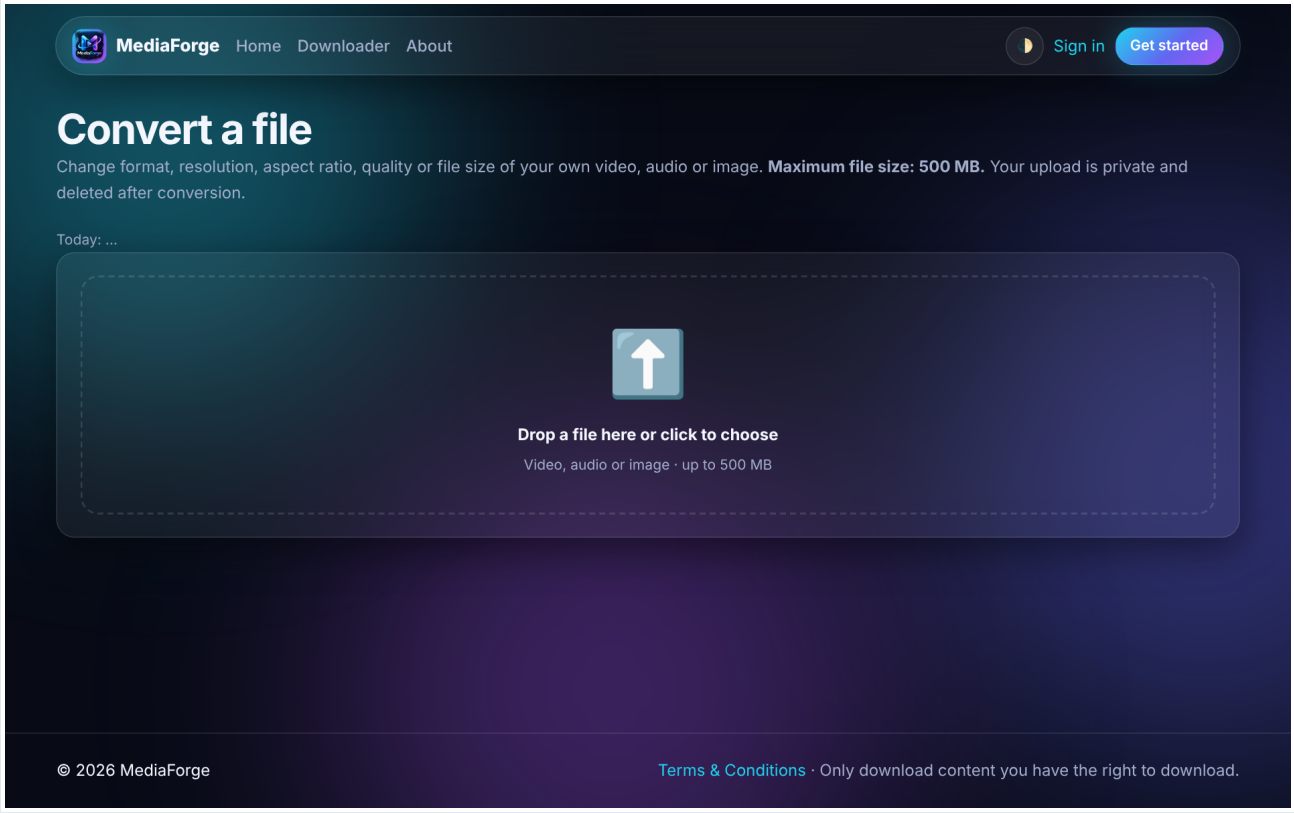
Downloader page. After a link is pasted, qualities appear in ascending order with the best one last, each with its estimated size.

- Platform is detected from the link; unsupported links are rejected with a clear message.
- Video qualities are real picture heights, listed from smallest to best, and every option shows an estimated size.
- Audio-only output is available for supported sources, with MP3 and M4A choices.
- Progress shows stage and percentage; when the server is busy it shows "You are #N in the queue for Download" and starts automatically when a slot frees up.

PRODUCT TOUR

Converter

Upload your own file and change its format, size or quality. The upload is private and removed after conversion.



Converter upload area. The limit shown (500 MB here) is the signed-in user's tier limit for conversions; admins see "any size".

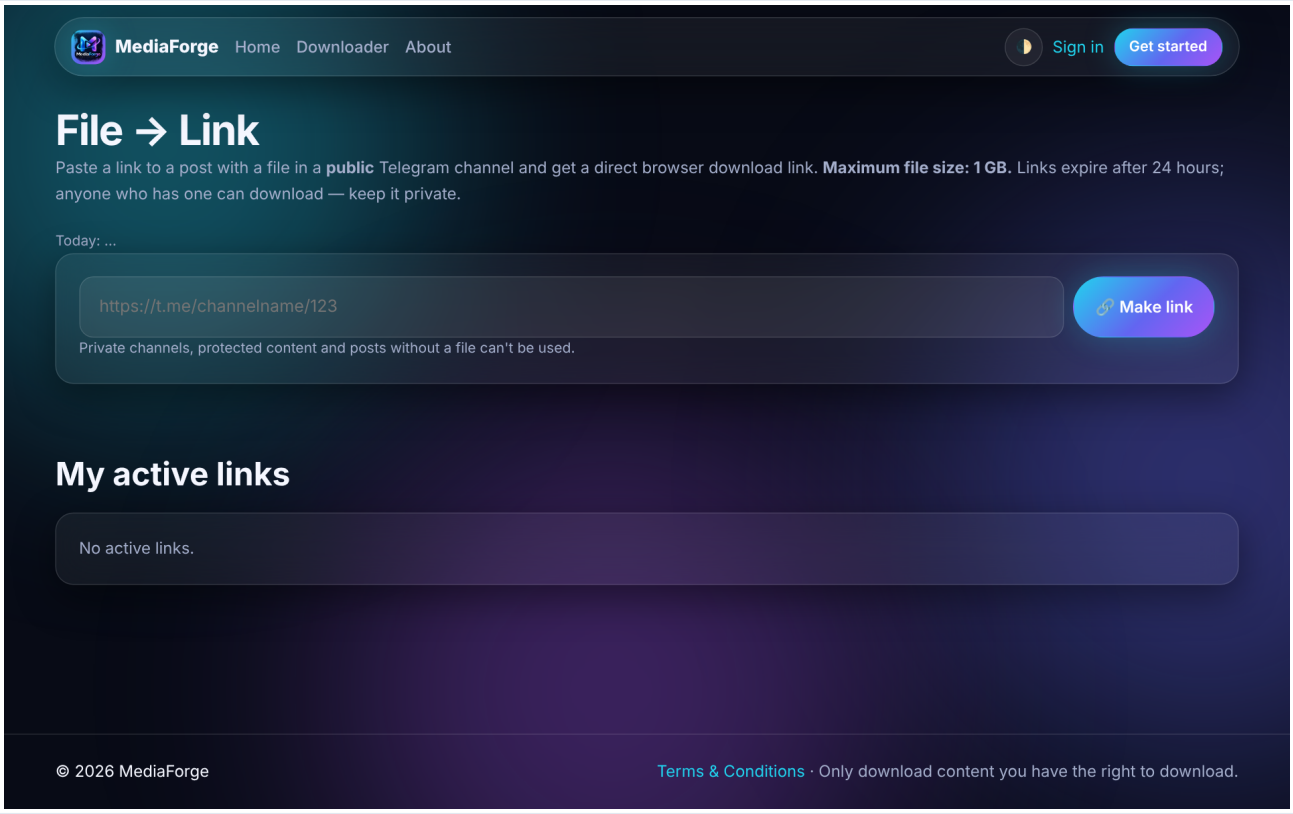
Kind	Options offered
Video	Output container, resolution from 240p up to 2160p, aspect ratio, frame rate, audio bitrate or a target file size.
Audio	Output format and bitrate.
Image	Output format, width from 320 to 3840 pixels and quality.

Conversions run on a separate lane of the job manager so a burst of FFmpeg work can never occupy every worker and block plain downloads.

PRODUCT TOUR

File to link

Turn a file into a temporary, shareable link without making the recipient sign in.

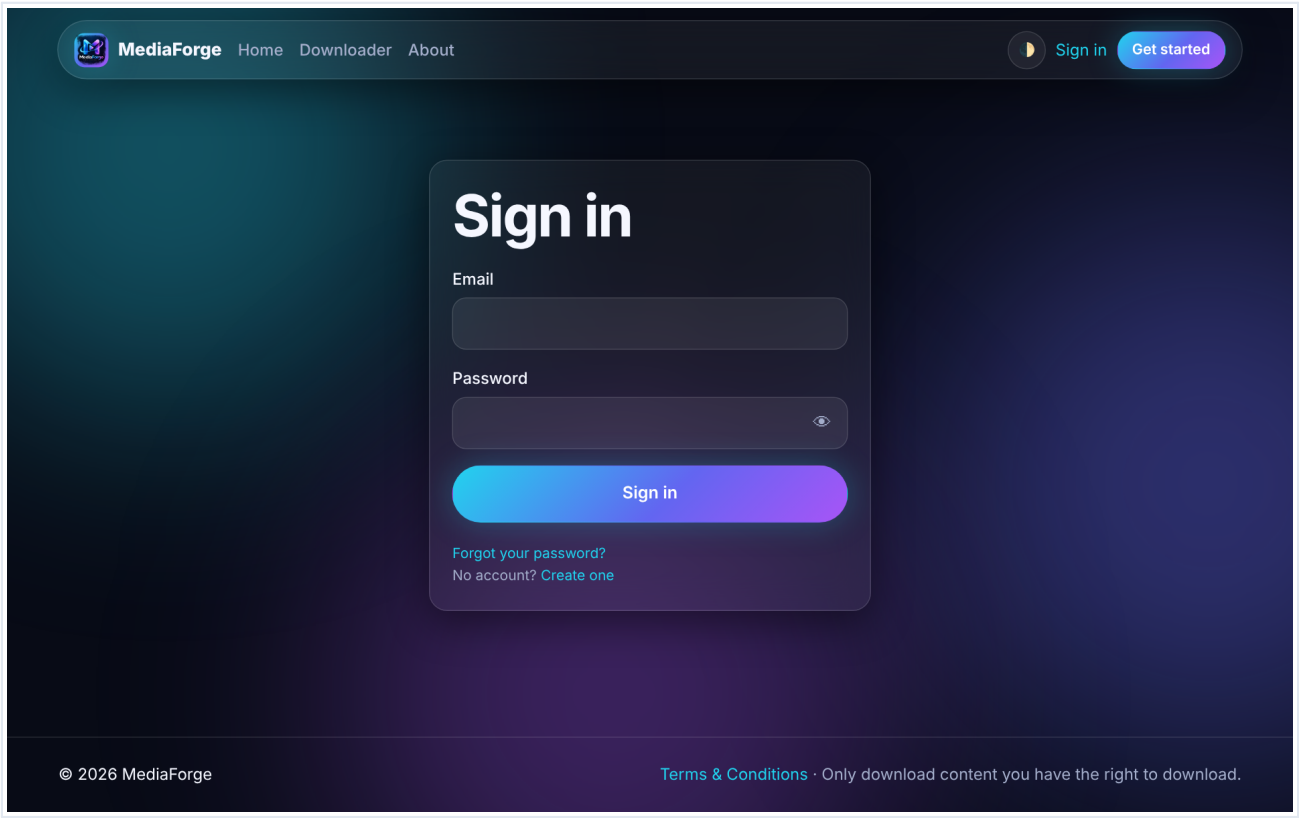


File to link. The file is stored in the private Telegram channel and served through the app until the link expires.

Links carry an expiry (24 hours by default) and are served by the application through a streaming endpoint with its own concurrency cap, so a popular link cannot starve the rest of the site. When the link expires, the stored file is scheduled for deletion.

PRODUCT TOUR

Accounts and sign-in



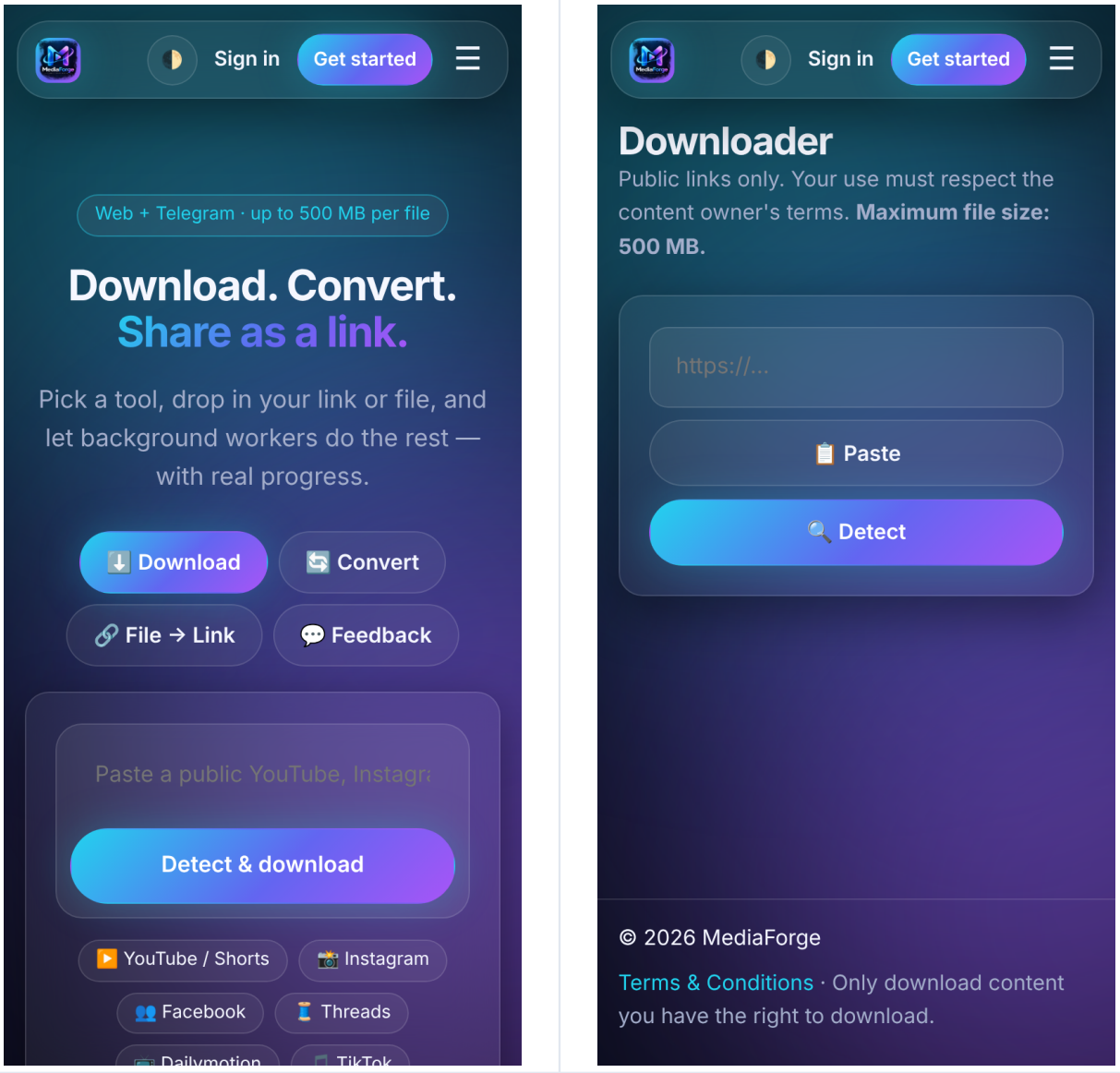
Sign-in screen. Registration verifies email ownership; password reset uses single-use hashed tokens.

Registration verifies email ownership before an account can be used, and password reset uses single-use tokens that are stored only as hashes. Signed-in users also get a history page, settings (including linking a Telegram account) and a feedback form.

PRODUCT TOUR

Mobile layout

The same pages reflow for a 390 px wide phone.



Home and downloader on a phone-sized viewport. Controls stack, targets stay thumb-sized.

- A single responsive stylesheet set, with no front-end framework: plain HTML templates and small JavaScript modules.
- Theme choice is remembered per browser and respects the system preference.
- An optional developer-tools guard can be switched on for non-admin visitors; it is a deterrent, not a security boundary.

PRODUCT TOUR

The Telegram bot

Everything is a button flow. Nobody has to memorise a command with arguments.

The bot uses the same services as the website. Sending a link starts a download; the bot replies with quality buttons in ascending order, shows progress by editing one message, and delivers the finished file. Admin actions that used to be long commands are now guided screens.

Flow	How it works
Download	Send a link, pick audio or video, pick a quality button (size shown), watch progress, receive the file.
Convert	Send a file, choose an output kind and options with buttons.
File to link	Send a file, receive a temporary link with its expiry in IST.
Link account	A short code connects the Telegram identity to the website account so both share one pool of daily uses.
Queue feedback	While waiting, the message reads "You are #N in the queue for Download" and updates as the line moves.
Admin	Give or revoke Premium and VIP, extend access, view stats, bans, load and maintenance, all by buttons with a confirm step.

> **Design choice.** Admin access changes are a three-step flow: choose the action, send the target person, confirm. The confirm screen prevents the most common admin mistake, which is applying a change to the wrong account.

Why a bot and a website

Telegram is where many of the target users already are, and it provides a convenient private store for large files. The website serves people who prefer a browser, and it is the surface a recruiter or reviewer can open immediately. Sharing one core means the two cannot drift apart in behaviour.

PRODUCT DESIGN

Tiers, limits and fairness

Three membership levels, one pooled daily allowance, and clear messages whenever a limit applies.

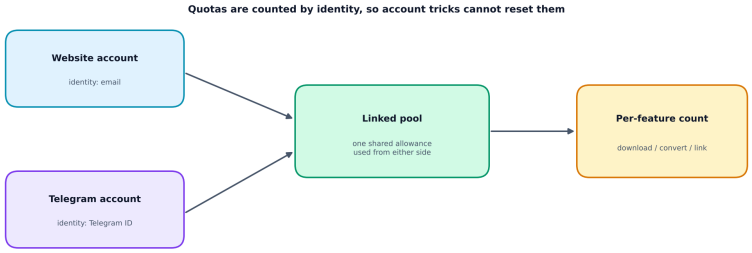
Tier	Daily uses per feature	Default file cap	Queue priority
Normal	5	500 MB	3 (default)
Premium	10	1 GB	2
VIP	20	2 GB (more with a Telegram Premium storage session)	1
Admin	exempt	no limit	-1 (always first)

Limits are configuration, not code: each value comes from an environment variable, so the operator can tune them to the host's disk and bandwidth without a redeploy of logic. Features are counted separately: downloads, conversions and file links each have their own allowance.

Per-feature size caps

A file-size cap exists for each feature (download, convert, link) and for each tier. The cap is checked early from the reported size and again as the file arrives, so a source that under-reports its size still cannot exceed the limit. Admins are exempt, which the interface states with "of any size" wording.

One pool across web and Telegram



A linked account shares one pool, counted by identity so unlinking and re-registering cannot refill it.

- Separate accounts each get their own allowance. A linked account gets a pooled allowance usable from either side.
- Linking is allowed only when nothing has been used today, and unlinking only from the next day, which closes the obvious loophole.
- Failed or cancelled jobs are not counted. The day runs midnight to midnight in IST.
- An admin can grant a one-day bonus that lapses at the next midnight.

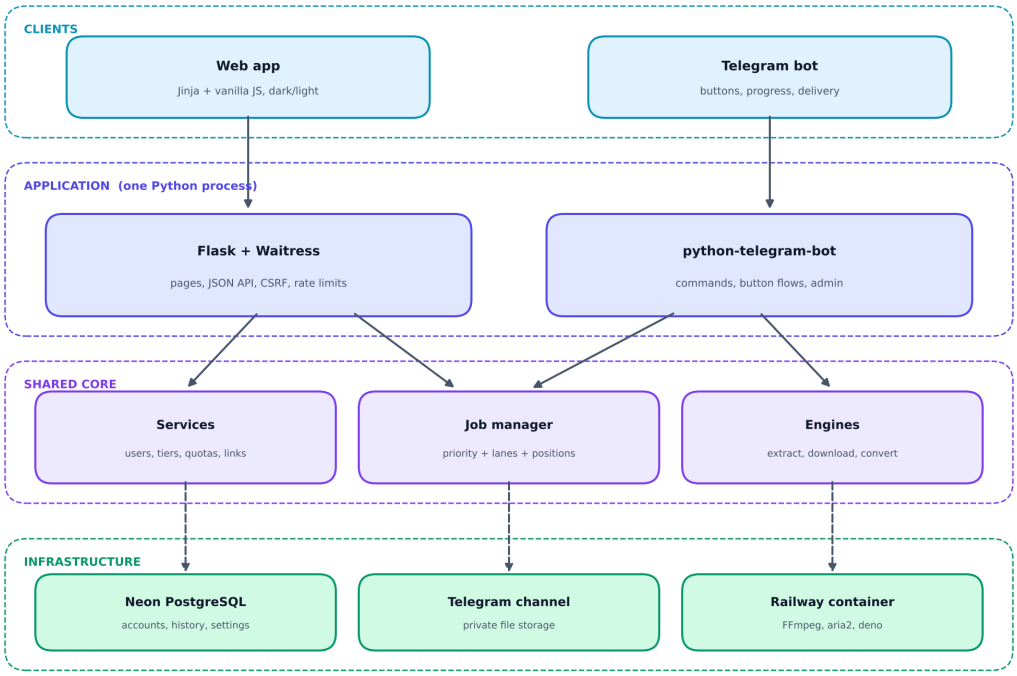
Rate limiting versus quotas

Two different mechanisms exist on purpose. Rate limits protect the server from bursts (requests per minute on sensitive endpoints) and return a friendly page that explains what was limited. Quotas express the product's usage policy (uses per day). Mixing the two usually produces confusing errors, so they are separate code paths with separate messages.

ENGINEERING

Architecture

One process, three kinds of threads, and a strict separation between interfaces, services and engines.



System architecture. Both interfaces call the same services; heavy work happens in engines behind the job manager.

Layer	Responsibility	Rule it follows
Interfaces	Flask pages and JSON API; Telegram commands and buttons	No business rules here: validate, call a service, render.
Services	Users, tiers, quotas, links, bans, history, backups, settings	All decisions about who may do what live here.
Job manager	Ordered queue with priorities, lanes and positions	Nothing is rejected only because the server is busy.
Engines	Platform detection, extraction, download, conversion, storage	Pure-ish functions that report progress and raise typed errors.
Database	SQLAlchemy models over SQLite (dev) or PostgreSQL (prod)	Stores UTC. Display conversion happens at the edge.

Process model

- **Main thread:** the Telegram bot's polling loop, which also handles shutdown signals.
- **Web thread:** Waitress serving the Flask application.
- **Worker threads:** the job manager's pool, split into lanes.
- **Background workers:** temp-file cleanup, scheduled deletions and maintenance claims.

If the bot token is missing or invalid, the website keeps running on its own, which also makes local development simple.

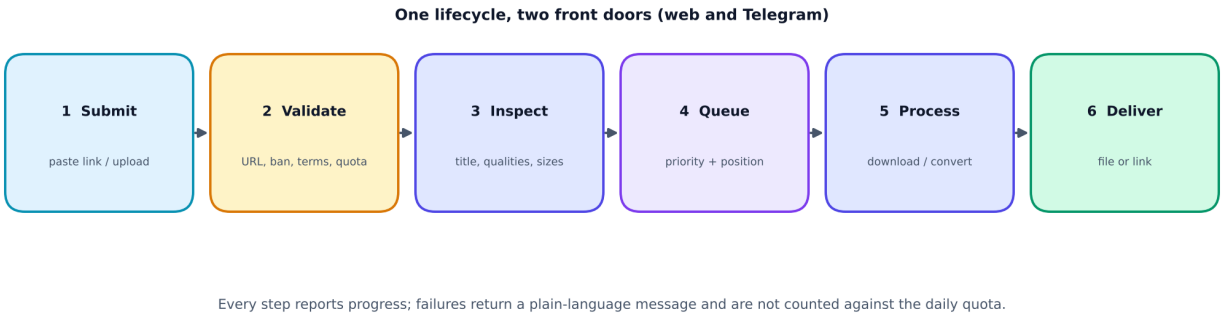
Why not microservices

A single process is the right size for a small team and a small host. The boundaries that matter, such as the job manager interface, are written so they can be swapped (for example for a Celery-style broker) without touching the callers. Choosing the simpler deployment and documenting the upgrade path is a deliberate trade-off.

ENGINEERING

Request lifecycle

The same six steps for every kind of work, from either interface.



Lifecycle of a job. Each stage reports progress and each failure maps to a user-facing message.

What happens at each step

Step	Details
Submit	A link or an upload arrives from the website or from Telegram.
Validate	URL syntax and network safety, ban list, terms acceptance, tier and quota are checked before any work starts.
Inspect	Metadata is fetched (and cached briefly) to show title, thumbnail, qualities and sizes.
Queue	The job gets a priority from the user's tier and a lane from its type; its position is exposed to the UI.
Process	A worker reserves disk space, downloads or converts, and reports progress.
Deliver	The file is stored, sent to the user or linked, and temporary files are released.

Error philosophy

Low-level failures are translated into a small set of typed errors, such as a source refusing the request, a login being required, a file being too large, or the network failing. Each type carries a user-facing message and a flag that says whether it counts toward the daily limit. This keeps raw exception text out of the interface and makes the support story consistent.

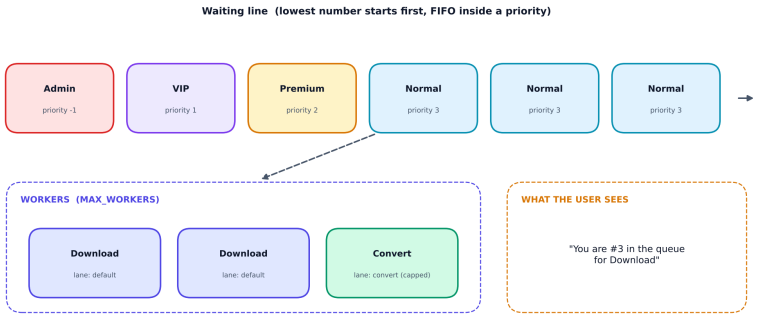
```
try:
    result = engine.run(job)           # download or convert
except SourceBlockedError as err:      # typed error, not a raw exception
    job.fail(err.user_message, counts=False)
```

Shape of the error handling (illustrative, not the actual source).

ENGINEERING

Queue and priority

A small scheduler written for this project, because the product needs visible positions and lanes.



Priority queue with lanes. Lower numbers start first; inside the same priority, first come first served.

The scheduler replaces a plain thread pool. A pool would run work in arrival order and could not tell a waiting user where they stand. The custom manager keeps a pending list sorted by a two-part key of priority and arrival sequence, runs a fixed number of workers, and wakes a worker whenever a slot frees up.

```
sort_key = (priority, arrival_sequence)    # admin -1, VIP 1, Premium 2, Normal 3
# a worker takes the best waiting item whose lane still has room
```

The ordering idea in two lines.

Lanes

Each job names a lane. Conversions use a lane with its own, smaller limit, so FFmpeg-heavy jobs cannot take every worker, and downloads cannot starve conversions entirely either. The lane limit is configuration.

Positions and messages

The manager reports a 1-based position for any waiting job. The service turns it into a sentence that names the function, such as downloading or converting, and both interfaces show the same wording.

Behaviour that is tested

Tests cover priority ordering, first-come-first-served inside a priority, cancelling queued and running jobs, and a lane limit that holds back extra conversions while downloads continue.

> **Honest limitation.** The queue lives in the process. It is correct for one replica; running several replicas would need an external broker. The interface to the manager is narrow so that change stays local.

ENGINEERING

Protecting the host

A small disk is a hard limit. The design treats space as a resource to reserve, not a hope.



Reservations stop two big jobs from filling a small disk at the same time.

Disk reservation flow. Space is claimed before a download starts and always released afterwards.

- **Temp ceiling:** total temporary usage is capped separately from the volume's free space.
- **Minimum free space:** jobs wait, rather than fail, when free space is below a configured floor.
- **Reservations:** an estimated size is claimed up front, so two large jobs cannot both assume the same free space.
- **Merge overhead:** video plus audio merging needs about twice the final size, and the guard accounts for that.
- **Can ever fit:** a job that could never fit even on an empty disk fails immediately with a clear message instead of waiting forever.
- **Always release:** the reservation is released in a finally block, including on cancellation and error.

A waiting job polls for space until a configured timeout, then fails politely. This turns a class of random out-of-space crashes into an understandable queue message.

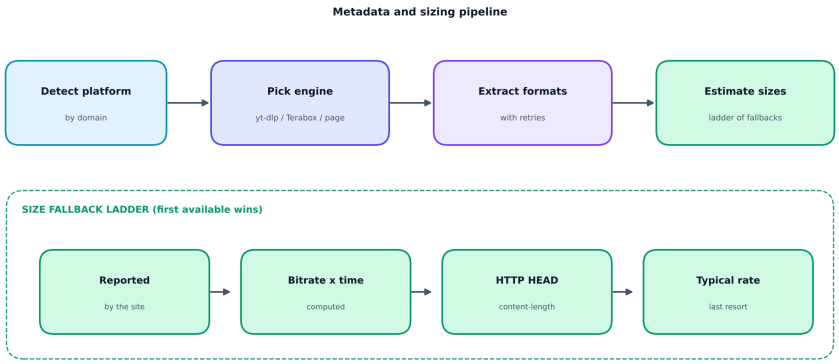
Cleanup

A cleanup worker removes temporary files on a schedule, and a purge runs at startup to clear leftovers from a previous crash. Finished files are not kept on the server disk: they are backed up to the private Telegram channel and the local copy is deleted on a retention timer.

ENGINEERING

Extraction and sizing

Find what can be downloaded, show real choices, and tell the user how big each one is before they commit.



Metadata pipeline with a size fallback ladder.

Engines

Engine	Used for	Notes
yt-dlp	Most video and audio platforms	Kept current with a scheduled rebuild; aria2 and parallel fragments for speed.
Terabox adapter	Terabox share links	Supports many mirror domains from one maintained list.
Open-graph adapter	Threads posts	Reads the public page's video address, then hands the direct file to the normal downloader.

Quality list

Qualities are real picture heights, shown in ascending order with the best last. Vertical videos are labelled by their short side so a vertical clip is not mislabelled. Each entry carries a size so the user can pick knowing what it costs.

Size estimation ladder

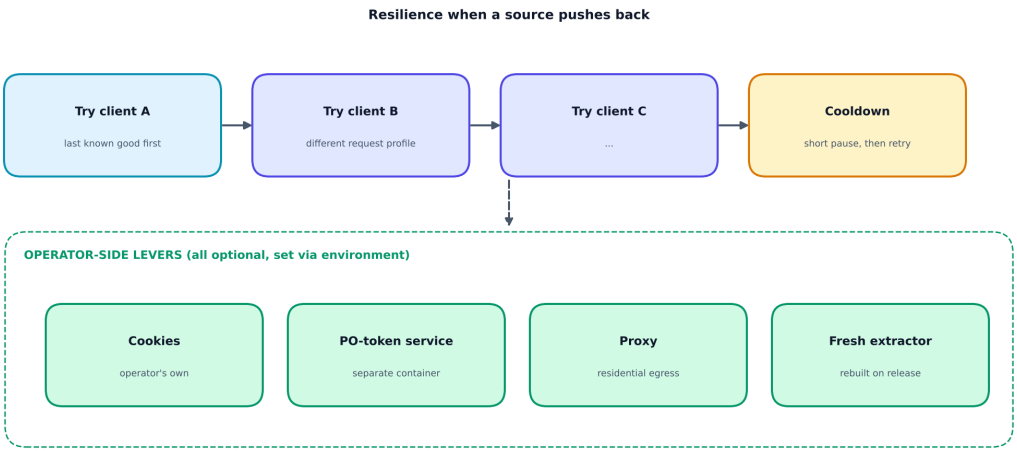
- Use the size the site reports if it does.
- Otherwise compute bitrate multiplied by duration.
- Otherwise send a lightweight HEAD request and read the content length.
- Last resort: a typical bitrate for that resolution, labelled as an estimate.

Sizes are also checked against the user's tier limit before the job is queued, so a request that cannot succeed is stopped in the first second, not after a long download. Metadata results are also cached for 15 minutes, which speeds up the interface and reduces repeated calls to a source.

ENGINEERING

Source reliability

Platforms change and sometimes refuse automated traffic. The design assumes this and degrades gracefully.



Retry chain with optional operator-side levers.

The hardest operational problem in this kind of product is not code. Video platforms often challenge requests that come from datacenter addresses. A deployment can work for hours and then start returning a bot check. MediaForge handles this in layers.

What the code does

- **Client fallbacks:** several request profiles are tried in turn, with the last successful one tried first next time.
- **Selective profiles:** profiles that need extra proof tokens are skipped unless a token service is configured.
- **Concurrency gate:** only a few lookups per source run at once, with a small gap between them.
- **Cooldown:** after a challenge, new lookups pause briefly rather than hammering the source.
- **Diagnostics:** the source's own explanation is logged on failure, so the next fix is based on evidence.
- **Cheap keep-alive:** uptime pings answer with an empty response and never touch a source.

What the operator can add

Lever	What it provides	Trade-off
Operator's own cookies	An authenticated session for the source	Use a throwaway account; cookies expire.
Proof-token service	A separate container that generates the tokens some profiles require	One more service to run.

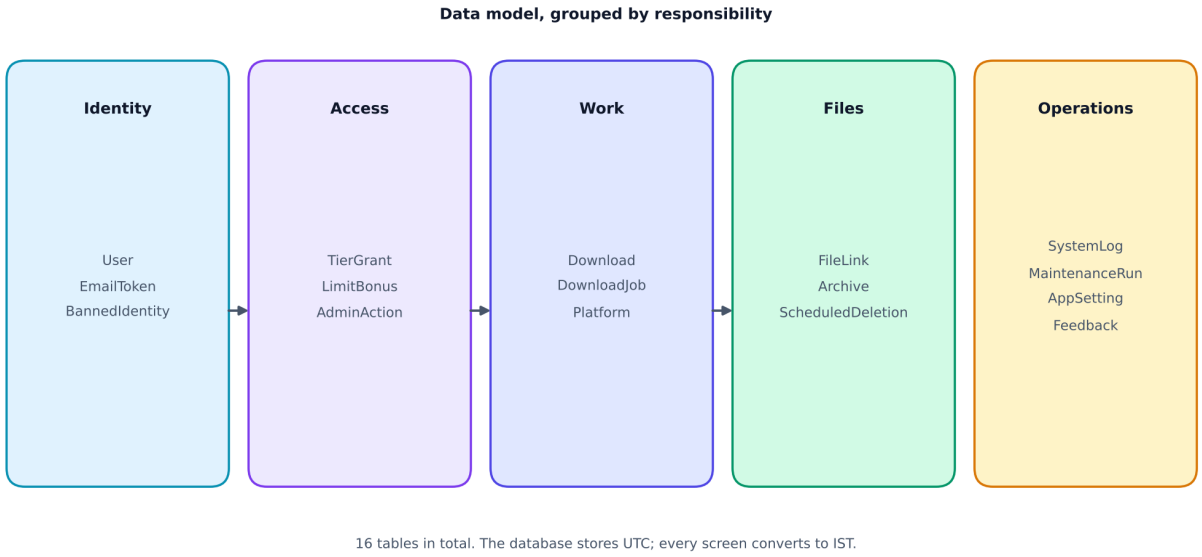
Lever	What it provides	Trade-off
Proxy	Egress from a residential address	Costs money; must be trustworthy.
Fresh extractor	A scheduled workflow rebuilds the image when a new extractor release appears	Rebuilds restart the service.

Candid status. Code alone cannot make a datacenter address look like a home connection. The project reduces failure and explains it, but reliability for the most guarded source depends on the operator choosing one of the levers above. The document states this plainly because it is the kind of limit a production owner has to know.

ENGINEERING

Data model

Sixteen tables grouped by what they are responsible for.



Data model groups.

Group	Purpose
Identity	Accounts, single-use email tokens (stored hashed) and banned identities.
Access	Tier grants with expiry, one-day limit bonuses, and an audit trail of admin actions.
Work	Download records, job state for progress and queue info, and the platform catalogue.
Files	Shareable links, archived files in Telegram storage and scheduled deletions.
Operations	System logs, maintenance run claims, runtime settings and user feedback.

Decisions worth noting

- **UTC in the database, IST on screen.** A fixed +05:30 offset is exact for India, needs no time-zone database and keeps backups unambiguous.
- **Identity-based counting.** Usage is keyed by identity, not by account row, so a deleted or re-created account does not reset a quota.
- **Maintenance claims.** Hourly and daily jobs record a (kind, period) row first, so restarts and overlapping runs do not repeat the work or raise duplicate-key errors.
- **SQLite for development, PostgreSQL for production.** The same models run on both; the production database is a managed pooled connection.

ENGINEERING

Storage and delivery

Large files live in a private Telegram channel instead of the application's small disk.

When a job finishes, the file is uploaded to a private channel through a user-level Telegram client. This permits large files and removes the need for a bucket. The server's own copy is deleted after a short retention window.

Aspect	Approach
Large files	Uploaded via the Telegram client library; an accelerated crypto module speeds up transfers.
Re-use	An identical request can be served from the backed-up copy without downloading again (cache setting).
Links	The app streams the stored file to the recipient; the number of simultaneous streams is capped.
Expiry	Links expire; expired files are queued for deletion by a scheduled worker.
Failure	If storage cannot start, new downloads are refused with a message and the admins are notified.

Streaming safety

The streaming endpoint has a concurrency ceiling. When it is reached, visitors see a friendly message that the service is handling many active streams, not a bare server error.

ENGINEERING

Security and privacy

Defence in depth, with the limits of each defence stated honestly.

Risk	Mitigation
Server-side request forgery	Every user-supplied URL is validated: only http(s), no embedded credentials, only standard ports, blocked internal names, and the host must resolve to a public address. Redirect targets on adapters are restricted to the platform's domains.
Cross-site request forgery	CSRF tokens on state-changing forms and API calls.
Credential storage	Passwords hashed with a salted algorithm; a dummy hash equalises timing so login does not reveal whether an email exists.
Token theft	Email and reset tokens are stored only as hashes and are single use.
Session safety	HttpOnly, SameSite=Lax cookies, Secure and host-prefixed in production, with a 7-day lifetime.
Browser attacks	Content security policy limited to the site's own origin, no framing, and no content-type sniffing.
Abuse	Per-route rate limits, daily quotas, bans keyed by identity and a flood guard on the bot.
Path and name tricks	Filename sanitising, safe path joins and Windows-reserved name handling.

Privacy stance

- Uploads for conversion are private and deleted after processing.
- History can be limited by policy and users can see what is stored about them.
- Secrets (bot token, API keys, cookies) come only from environment variables and are never written to the repository.
- The terms require acceptance and restrict use to content the user has the right to download.

Known boundary. The URL validator checks DNS before handing a link to the extractor, but the extractor resolves DNS again. A determined rebinding race is theoretically possible, so the deployment guidance is to run with restricted outbound networking as defence in depth.

OPERATIONS

Admin and operations

Everything an operator needs lives inside the product.

Capability	Where
Dashboard with totals, recent downloads, recent logs and live queue load	Web admin and bot admin
Give or revoke Premium or VIP, extend access, add a one-day bonus	Button flow with confirm step
Ban and unban identities	Web admin and bot
Maintenance mode and runtime settings	Web admin
Feedback inbox with status	Web admin
Database backup and restore	Web admin and a restore script
Notify admins on storage failures	Bot message

Administrators declared in the environment are permanent, so a database mistake cannot lock the owner out. Admin actions are written to an audit table with who, what and when in IST.

Observability

- Structured logs with IST timestamps, optionally mirrored into the database for the admin console.
- Health endpoint for liveness and a readiness endpoint that checks the database.
- Uptime pings are answered by the cheapest possible handler and skipped in the request log, which was a real fix after reading production logs.

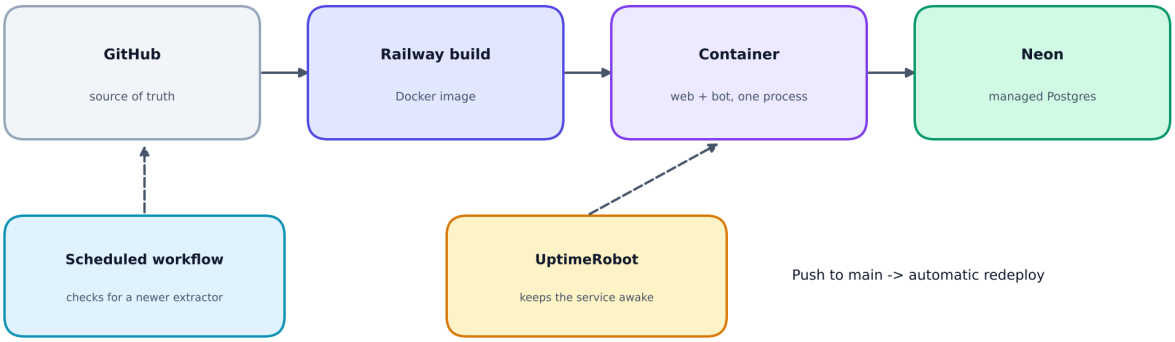
Maintenance

Hourly and daily tasks, such as expiring tier grants, purging expired links and deleting old files, are claimed through a unique (kind, period) record so only one run proceeds even after a restart.

OPERATIONS

Deployment

Push to GitHub, Railway builds the image, and the service restarts with a managed database.



Delivery path and the two helper automations.

- **Docker image:** Python runtime plus FFmpeg, aria2 and a JavaScript runtime that the extractor uses for some sources.
- **Configuration:** everything from environment variables, with a template file that documents each one.
- **Managed Postgres:** a pooled connection string suitable for a serverless database.
- **Extractor freshness:** a scheduled workflow compares the newest extractor release with a marker file and commits a change, which busts the Docker cache and reinstalls the latest version.
- **Keep-alive:** an external uptime monitor pings a cheap endpoint.

Setup check

A setup script verifies configuration before first run, such as required variables, database reachability and storage credentials, and prints what is missing in plain language.

QUALITY

Testing and verification

Tests focus on rules that are easy to break and expensive to debug in production.

Area	What is checked
Job manager	Priority order, FIFO inside a priority, cancel behaviour, lane limits.
Tiers and limits	Tier limits, admin exemption, pooled quotas, link and unlink rules.
Sizing	Estimation ladder, ascending quality labels, vertical video labelling.
Validation	SSRF cases, filename safety, URL normalisation.
Errors	Mapping of low-level messages to typed, user-facing errors.
Platforms	Detection by domain, including Terabox mirrors, Threads and Dailymotion.
Conversion	Option sets and ordering for video, audio and image.
Permissions and accounts	Who may see or change what; registration and verification rules.

The pure-logic tests run anywhere without a network. A second group exercises the database through SQLAlchemy and needs the full dependency set, so they run in a complete environment rather than in a minimal sandbox.

> **What has not been done.** The project has not been load-tested, and the Threads adapter depends on public page markup that can change. These are listed again under Limitations so nobody is surprised.

REFLECTION

Problems found and how they were solved

Production logs and user screenshots drove several of the best changes.

Symptom	Cause	Change
Hourly database error in logs	Maintenance task inserted a duplicate (kind, period) row.	Check for an existing claim first, so no exception is raised or logged.
Heavy load from health pings	Monitor sent full page requests for the home page.	Empty responses for HEAD pings, skipped in request logging.
Download list in confusing order	Qualities listed best first, sizes missing for some.	Ascending order with best last and a size on every option.
Busy server rejected work	A plain pool had no waiting line.	Priority queue with lanes and visible positions.
Out-of-space failures	Two large jobs assumed the same free space.	Disk reservations with wait-or-fail rules.
Admin commands too long	One command carried every argument.	Button flows with a confirm step.
Source refusals after some hours	Datacenter address challenged by the source.	Client fallbacks, cooldowns, diagnostics and operator levers.

What I would do next

- Move the queue and rate-limit storage to an external broker so more than one replica is safe.
- Add end-to-end tests with recorded source responses.
- Add metrics export and a dashboard for queue depth and failure rates.
- Add a public status page driven by the existing readiness endpoint.

ENGINEERING

Design decisions

The choices that shaped the system, with the reasoning and the price paid for each.

Decision	Why	Trade-off accepted
Single process for web, bot and workers	Simplest thing that fits a small host; one deploy, one log stream.	Cannot scale horizontally without moving shared state out.
Custom job manager instead of a plain thread pool	Needed priorities, lanes and visible queue positions.	More code to own; covered by tests.
Telegram channel as file store	Large files without a bucket and without the small disk.	Depends on a third-party service; one more failure mode, handled with admin alerts.
UTC in the database, IST at the edge	Unambiguous storage and a local experience.	Every display path must go through the formatting helpers.
Identity-based quotas	Resets by account tricks are not possible.	Linking and unlinking need explicit rules.
Typed errors with user messages	Consistent support story and no raw exceptions on screen.	Mapping table must be kept up to date as sources change.
No front-end framework	No build step, small payloads, easy to read.	More manual DOM code in the larger pages.
Reservations instead of hoping for free space	Prevents out-of-space crashes under concurrency.	A job may wait; the message explains why.

A rule I followed

Where a decision could go either way, I chose the one that fails visibly and politely: wait in a line, show a reason, release resources, and keep the other users unaffected.

ENGINEERING

One download, end to end

A walk-through of what happens between pressing the button and receiving the file.

#	What the user sees	What the system does
1	Pastes a link and presses Detect and download.	The platform is recognised from the domain. The address is checked for network safety.
2	A short lookup state.	Metadata is fetched through the right engine, with retries and a cache. Sizes are estimated for each quality.
3	A list of qualities in ascending order, best last, each with a size.	Sizes above the user's tier limit are flagged before any work is queued.
4	Chooses a quality and starts.	Terms, ban status and the day's quota are checked. A job record is created.
5	"You are #2 in the queue for Download" if busy.	The job waits in priority order. Disk space is reserved when it reaches a worker.
6	Progress bar with stages.	The download runs with parallel connections; merge and conversion steps report progress.
7	The file arrives, or a link appears.	The file is backed up to storage, delivered, counted against the quota, and the temporary copy is released.
8	If anything fails, a clear message.	The error is mapped to a typed message. Failed jobs are not counted.

Why this matters. Every row has a visible state for the user and a defined resource rule for the server. That pairing is the core idea of the project.

REFERENCE

Technology stack

Layer	Technology	Why
Language	Python 3.12	Fast to build, strong ecosystem for media tooling.
Web	Flask, Waitress, Flask-WTF, Flask-Limiter	Small, explicit, easy to reason about.
Bot	python-telegram-bot	Mature async library with button support.
Storage client	Telethon with a native crypto module	User-level API for large files.
Database	SQLAlchemy 2, SQLite (dev), PostgreSQL on Neon (prod)	One model set, two environments.
Media	yt-dlp, FFmpeg, aria2	Best-in-class extraction, conversion and parallel download.
Front end	Jinja templates, vanilla JavaScript, custom CSS	No build step, fast first paint.
Delivery	Docker, Railway, GitHub Actions	Push-to-deploy with an automated freshness job.
Monitoring	Health and readiness endpoints, UptimeRobot, structured logs	Cheap and effective for a small service.

REFERENCE

Glossary

Short definitions for terms used in this document.

Term	Meaning
Extractor	A library that reads a page and works out which media files exist and how to fetch them.
Lane	A named group of jobs with its own concurrency limit, so heavy work cannot take every worker.
Priority	A number attached to a job; lower numbers start first.
Quota	How many times per day a user may use a feature. Separate from rate limiting.
Rate limit	A cap on how fast requests may arrive, to protect the server from bursts.
Reservation	A claim on disk space made before work starts and released afterwards.
SSRF	Server-side request forgery: tricking a server into requesting internal addresses.
CSRF	Cross-site request forgery: a site making a logged-in browser submit an unwanted action.
IST	Indian Standard Time, UTC+05:30, with no daylight saving.
Pooled quota	One shared daily allowance for a website account and a linked Telegram account.
Proof token	Extra data some sources require from a client before serving media.
Readiness endpoint	A URL that confirms the app and its database are reachable.

REFERENCE

Limitations, stated plainly

Knowing where a system stops is part of building it well.

- **Single replica.** The queue and rate limit counters live in memory. Scaling out needs a broker.
- **Source dependence.** Platforms can change or refuse automated traffic. The project mitigates this but cannot remove it.
- **Not load-tested.** Behaviour under heavy concurrency is designed and unit-tested, not benchmarked.
- **Threads adapter.** It relies on markup that the site can change at any time.
- **Large-file path.** Files above the standard Telegram limit need a Telegram Premium storage session; that route is the least exercised.
- **Legal use.** The tool is intended for content the user has the right to download. It does not bypass logins, DRM or paywalls.

Where the real numbers will come from

This document deliberately contains no throughput, latency or user-count claims. Those belong in measured results from the production deployment. Once collected, queue wait time, success rate by platform and download throughput would be the three numbers that best describe the system.

FOR REVIEWERS

Skills this project demonstrates

Skill	Evidence in the project
Backend design	Layered architecture, narrow interfaces, replaceable job manager.
Concurrency	Custom priority scheduler with lanes, cancellation and position reporting.
Resource management	Disk reservations, bounded concurrency, graceful waiting.
Data modelling	Sixteen tables, identity-based quota counting, audit trail, safe maintenance claims.
Security	SSRF-aware validation, CSRF, hashed tokens, timing-safe login, strict headers.
Product thinking	Visible queue positions, sizes before download, plain-language errors.
Operations	Docker, managed database, health checks, backups, automated dependency freshness.
Debugging from evidence	Fixes driven by real production logs and user screenshots.
Integration	Web, Telegram bot, external extractors, FFmpeg and a managed database working together.
Honest engineering	Limitations documented; no unverified performance claims.

CLOSING

Thank you for reading

MediaForge started as a small downloader and grew into a study in making a service fair, safe and understandable. The part I am proudest of is not any single feature. It is that a person using it can always tell what the system is doing, and a person running it can always tell what went wrong.

I would welcome the chance to talk through any part of the design: the scheduler, the quota rules, the way source failures are handled, or what I would change to scale it.

Contact	
Name	Abhay Sharma
Email	movieznagar@gmail.com

> **Responsible use.** MediaForge is for content you have the right to download. Respect copyright and each platform's terms of service.